

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**



ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И УПРАВЛЯЮЩИХ СИСТЕМ

Лабораторная работа №1
по дисциплине: **Операционные системы**
тема: **«Базовая работа с процессами в ОС Linux. Сигналы»**

Выполнил: ст. группы ВТ-231
Масленников Д. А.

Проверили:
Островский А. М.

Белгород 2025 г

Лабораторная работа № 1

Цель работы: Изучить основы работы с процессами в ОС Linux, а также освоить сигналы как

универсальное средство межпроцессного взаимодействия.

Цель работы обуславливает постановку и решение следующих **задач**:

1. Изучить основные понятия о процессах в ОС Linux, относящиеся к их созданию и управлению.
2. Ознакомиться с базовыми системными вызовами POSIX для работы с процессами: `fork()`, `wait()`, `waitpid()`.
3. Изучить назначение и возможности сигналов в Linux, их классификацию (системные, пользовательские) и обработку.
4. Освоить установку обработчиков сигналов с помощью `signal()` и `sigaction()`, а также способы отправки сигналов с помощью `kill()` и `sigqueue()`.
5. Познакомиться с соответствующими консольными инструментами для мониторинга и управления процессами и сигналами.
 - 5.1 Изучение команд `ps`, `top`, `htop` — просмотр списка процессов, их состояний и ресурсов.
 - 5.2 Изучение команд `jobs`, `bg`, `fg` — управление фоновыми и приостановленными задачами в оболочке.
 - 5.3 Изучение команд `kill`, `pkill`, `killall` — отправка сигналов процессам по PID или имени.
 - 5.4 Изучение команды `ulimit` — просмотр и установка ресурсных ограничений (например, число процессов, размер стека).
 - 5.5 Изучение команды `strace` — отслеживание системных вызовов процесса, включая работу с сигналами.
6. Выполнить индивидуальное задание для закрепления знаний на практике, подготовив соответствующую программу на языке C (номер задания соответствует номеру студента по журналу; если этот номер больше, чем максимальное число заданий, тогда вариант задания вычисляется по формуле: номер по журналу % максимальный номер задания, где % — остаток от деления; если остаток равен 0, студенту назначается задание с максимальным номером).
7. Подготовить отчёт по работе, который должен включать: краткое описание всех использованных системных вызовов и команд; текст программы; протоколы; скриншоты с демонстрацией поведения ОС Linux; выводы.

Вариант 3

3. Породить кольцо из N процессов «родитель 0 → потомок 1 → потомок 2 → ... → потомок N-1 → родитель 0». Осуществить передачу по кольцу целого числа S, которое печатается в консоль текущим процессом вместе со своим PID и PPID сразу после получения S. Перед передачей S далее по кольцу значение S увеличивается текущим процессом на единицу. Для передачи S можно воспользоваться, например, библиотечной функцией стандарта POSIX `sigqueue`.

Выполнение индивидуального задания:

Исходный код программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

#define N 4
#define SIG_RING (SIGRTMIN + 1)

static pid_t g_original_parent = -1;
static pid_t g_next_pid = -1;

void sigterm_handler(int signo) {
    exit(EXIT_SUCCESS);
}

void sigint_handler(int signo) {
    // Только исходный родитель обрабатывает SIGINT
    if (getpid() == g_original_parent) {
        printf("\n[Parent] Получен SIGINT. Завершение...\n");
        exit(EXIT_SUCCESS);
    }
}

void ring_handler(int signo, siginfo_t *info, void *context) {
    int value = info->si_value.sival_int;
    pid_t self = getpid();
    pid_t parent = getppid();

    printf("[PID=%d, PPID=%d] Получено: %d\n", (int)self, (int)parent,
value);

    // Увеличиваем значение
    int new_value = value + 1;

    // Отправляем дальше
    const union sigval sv = {.sival_int = new_value};
    if (sigqueue(g_next_pid, SIG_RING, sv) == -1) {
        perror("sigqueue forward");
        exit(EXIT_FAILURE);
    }
}

int main(int argc, char *argv[]) {
```

```

setvbuf(stdout, NULL, _IONBF, 0);

// Определяем PID исходного родителя
if (argc == 1) {
    // Это исходный родитель
    g_original_parent = getpid();
} else {
    // Это потомок – получаем PID родителя из аргумента
    g_original_parent = (pid_t)atol(argv[1]);
}

pid_t self_pid = getpid();

// Устанавливаем обработчики сигналов
struct sigaction sa = {0};
sa.sa_sigaction = ring_handler;
sa.sa_flags = SA_SIGINFO;
sigemptyset(&sa.sa_mask);
if (sigaction(SIG_RING, &sa, NULL) == -1) {
    perror("sigaction");
    exit(EXIT_FAILURE);
}

if (signal(SIGTERM, sigterm_handler) == SIG_ERR) {
    perror("signal SIGTERM");
    exit(EXIT_FAILURE);
}

// Только исходный родитель ловит SIGINT
if (self_pid == g_original_parent) {
    if (signal(SIGINT, sigint_handler) == SIG_ERR) {
        perror("signal SIGINT");
        exit(EXIT_FAILURE);
    }
}

// === ЦЕПОЧКА ПОРОЖДЕНИЯ ===
if (self_pid == g_original_parent) {
    // Родитель: создаёт первого потомка, если N > 1
    if (N > 1) {
        char orig_pid_str[32];
        snprintf(orig_pid_str, sizeof(orig_pid_str), "%ld",
(long)g_original_parent);
        pid_t first_child = fork();
        if (first_child == -1) {
            perror("fork");
            exit(EXIT_FAILURE);
        }
    }
}

```

```

    }
    if (first_child == 0) {
        execl(argv[0], argv[0], orig_pid_str, "1", NULL); // "1"
= уровень/номер
        perror("execl first child");
        exit(EXIT_FAILURE);
    }
    g_next_pid = first_child;
} else {
    // N == 1: кольцо из одного процесса (сам себе)
    g_next_pid = g_original_parent;
}
} else {
    // Это потомок. Определяем, сколько процессов уже создано.
    // Мы передаём "глубину" через argv[2]: 1, 2, ..., N-1
    int depth = (argc >= 3) ? atoi(argv[2]) : 1;

    if (depth < N - 1) {
        // Нужно создать следующего потомка
        char orig_pid_str[32];
        char next_depth_str[32];
        snprintf(orig_pid_str, sizeof(orig_pid_str), "%ld",
(long)g_original_parent);
        snprintf(next_depth_str, sizeof(next_depth_str), "%d", depth
+ 1);

        pid_t next_child = fork();
        if (next_child == -1) {
            perror("fork in child");
            exit(EXIT_FAILURE);
        }
        if (next_child == 0) {
            execl(argv[0], argv[0], orig_pid_str, next_depth_str,
NULL);

            perror("execl next child");
            exit(EXIT_FAILURE);
        }
        g_next_pid = next_child;
    } else {
        // Это последний потомок (глубина = N-1)
        // === ВОТ ЭТОТ МОМЕНТ – ЗАМЫКАНИЕ КОЛЬЦА ===
        // Последний процесс в цепочке отправляет сигнал
        // обратно исходному родителю, замыкая кольцо:
        // ... → Потомок N-1 → Родитель → Потомок 1 → ...
        // =====
        g_next_pid = g_original_parent;
    }
}

```

```

    }

    // Инициация передачи: только исходный родитель
    if (self_pid == g_original_parent) {
        sleep(1); // дать потомкам завершить инициализацию
        printf("[Parent %d] Иницирует передачу числа 0 в кольцо.\n",
(int)self_pid);
        const union sigval sv = {.sival_int = 0};
        if (sigqueue(g_next_pid, SIG_RING, sv) == -1) {
            perror("sigqueue init");
            exit(EXIT_FAILURE);
        }
    }
}

while (1) {
    pause();
}

return 0;
}

```

Результат выполнения программы:

```

[PID=39064, PPID=39063] Получено: 145766
[PID=39061, PPID=37324] Получено: 145767
[PID=39062, PPID=39061] Получено: 145768
[PID=39063, PPID=39062] Получено: 145769
^C
[Parent] Получен SIGINT. Завершение...

```

```

~/projects/university/5sem/os/lab1 master* > ps aux | grep ./res
he1nous+  37758  0.0  0.0  2576  1712 pts/0    S   23:10   0:00  ./res 37756
he1nous+  37759  0.0  0.0  2576  1716 pts/0    S   23:10   0:00  ./res 37756 1
he1nous+  37760  0.0  0.0  2576  1712 pts/0    S   23:10   0:00  ./res 37756 0
he1nous+  39517  0.0  0.0  6776  4248 pts/0    S+  23:18   0:00  grep --color=a

```

Вывод:

В ходе выполнения лабораторной работы №1 были изучены основы работы с процессами в ОС Linux, включая их создание с помощью `fork()`, синхронизацию и завершение через `wait()`. Освоены механизмы обработки сигналов с использованием `signal()` и `sigaction()`, а также отправка сигналов через `kill()` и `sigqueue()`. Практически реализовано межпроцессное взаимодействие по принципу «пинг-понг» (задание №9), продемонстрирована корректная обработка завершения по `SIGINT` и управление дочерними процессами. С помощью консольных утилит подтверждена работоспособность программы и изучено поведение процессов в реальном времени. Лабораторная работа позволила закрепить ключевые концепции управления процессами и сигналами в Linux.